

RESPONSIVIDADE EM DESENVOLVIMENTO WEB

1. Introdução à Responsividade

1.1 O que é responsividade?

Responsividade Web é a capacidade de uma página ou aplicação Web de adaptar seu layout, seus elementos e seu conteúdo a diferentes tamanhos e orientações de tela, proporcionando uma boa experiência de utilização em computadores, tablets e smartphones.

Uma página responsiva não deve simplesmente "diminuir" quando a tela fica menor. Seus elementos devem ser reorganizados de acordo com o espaço disponível.

Por exemplo, imagine uma página com três cartões.

Em uma tela grande:

```
+-----+ +-----+ +-----+
| CARD 1 | | CARD 2 | | CARD 3 |
+-----+ +-----+ +-----+
```

Em uma tela pequena:

```
+-----+
| CARD 1 |
+-----+

+-----+
| CARD 2 |
+-----+

+-----+
| CARD 3 |
+-----+
```

Portanto, responsividade significa fazer com que a interface acompanhe as características do dispositivo utilizado.

1.2 Por que a responsividade é necessária?

Uma página Web pode ser acessada utilizando diferentes dispositivos:

- computadores;
- notebooks;
- tablets;
- smartphones;
- televisores;
- monitores com diferentes resoluções.

Um layout criado exclusivamente para uma tela grande pode apresentar diversos problemas em uma tela pequena.

Entre os problemas mais comuns estão:

- conteúdo ultrapassando a tela;
- rolagem horizontal;
- textos muito pequenos;
- imagens cortadas;

- botões difíceis de utilizar;
 - menus ocupando espaço excessivo;
 - elementos sobrepostos;
 - espaços exagerados;
 - colunas que não cabem na tela.
-

2. Viewport

2.1 Conceito

Viewport é a área visível da página dentro da janela do navegador.

Em um computador, podemos ter uma viewport bastante larga:



Em um smartphone, a área disponível é muito menor:



O CSS pode utilizar o tamanho da viewport para adaptar os elementos da página.

3. Meta Viewport

Para que uma página HTML seja interpretada corretamente em dispositivos móveis, devemos utilizar a configuração:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Exemplo completo:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>Página Responsiva</title>
</head>
<body>
  <h1>Minha página</h1>
</body>
</html>
```

3.1 width=device-width

Determina que a largura da viewport deve corresponder à largura do dispositivo.

3.2 initial-scale=1.0

Define o nível inicial de zoom da página.

3.3 Importante

A meta viewport não torna a página responsiva sozinha.

Ela prepara o navegador para interpretar corretamente o layout em dispositivos móveis.

A responsividade será construída principalmente utilizando HTML e CSS.

4. Unidades de Medida

Um dos primeiros passos para criar páginas responsivas é compreender a diferença entre unidades absolutas e relativas.

4.1 Pixel — px

O px é uma unidade muito utilizada no CSS.

Exemplo:

```
.caixa {  
    width: 500px;  
}
```

Nesse caso, a largura será definida utilizando um valor fixo.

Em uma tela grande, isso pode funcionar bem. Porém, em uma tela pequena, uma largura fixa pode causar problemas.

Por exemplo:

```
+-----+  
|       |  
|  IMAGEM  |----->  
|       |  
+-----+
```

O elemento pode ultrapassar a largura disponível.

5. Porcentagem — %

A porcentagem é uma unidade relativa.

Exemplo:

```
.caixa {  
    width: 80%;  
}
```

Nesse caso, a caixa ocupará aproximadamente 80% da largura disponível do elemento pai.

Exemplo:

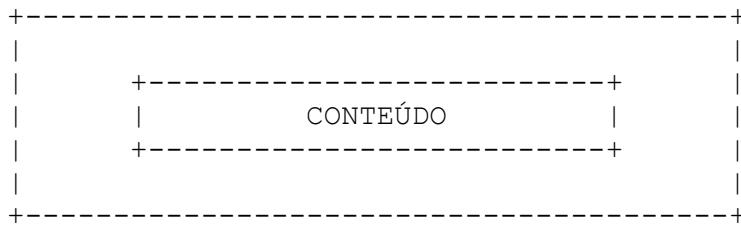
```
<div class="caixa">
  Conteúdo
</div>
.caixa {
  width: 80%;
  margin: auto;
}
```

A propriedade:

`margin: auto;`

pode centralizar horizontalmente um elemento que possui uma largura definida.

Visualmente:



6. Unidade em

A unidade em é relativa ao tamanho da fonte do elemento ou ao seu contexto de referência.

Exemplo:

```
.container {
  font-size: 20px;
}

.container p {
  font-size: 1.5em;
}
```

Nesse caso:

$$20\text{px} \times 1.5 = 30\text{px}$$

Portanto, o parágrafo terá aproximadamente 30px.

7. Unidade rem

A unidade rem é relativa ao tamanho da fonte do elemento raiz, normalmente o elemento `<html>`.

Exemplo:

```
html {
  font-size: 16px;
}

h1 {
  font-size: 2rem;
}
```

Nesse caso:

$$16\text{px} \times 2 = 32\text{px}$$

Portanto:

```
h1 {  
  font-size: 2rem;  
}
```

corresponde aproximadamente a 32px quando a fonte raiz possui 16px.

7.1 Exemplo

```
body {  
  font-size: 1rem;  
}  
h1 {  
  font-size: 2rem;  
}  
h2 {  
  font-size: 1.5rem;  
}  
p {  
  font-size: 1rem;  
}
```

O uso de rem facilita a criação de uma escala de tamanhos proporcional.

8. Unidade vw

vw significa **viewport width**.

Uma unidade vw representa 1% da largura da viewport.

Exemplo:

```
.titulo {  
  font-size: 5vw;  
}
```

Se a viewport tiver 1000px:

$$5\% \text{ de } 1000\text{px} = 50\text{px}$$

Se a viewport tiver 400px:

$$5\% \text{ de } 400\text{px} = 20\text{px}$$

Portanto, o tamanho acompanha a largura da tela.

9. Unidade vh

vh significa **viewport height**.

Uma unidade vh representa 1% da altura da viewport.

Exemplo:

```
.banner {  
  height: 50vh;  
}
```

Se a viewport tiver 800px de altura:

50% de 800px = 400px

10. Exemplo Prático com Unidades Relativas

HTML:

```
<div class="container">  
  <h1>Desenvolvimento Web</h1>  
  <p> Aprendendo a criar páginas responsivas.</p>  
</div>
```

CSS:

```
.container {  
  width: 80%;  
  min-height: 50vh;  
  margin: auto;  
  padding: 2rem;  
}  
h1 {  
  font-size: 3rem;  
}  
p {  
  font-size: 1.2rem;  
}
```

Nesse exemplo foram utilizadas:

- % para largura;
 - vh para altura;
 - rem para espaçamento;
 - rem para tamanho de fonte.
-

11. Imagens Responsivas

Um problema muito comum ocorre quando uma imagem possui largura maior que o espaço disponível.

Exemplo problemático:

```
img {  
  width: 800px;  
}
```

Se a tela possuir apenas 400px de largura, a imagem poderá ultrapassar o espaço disponível.

Uma solução comum é:

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

11.1 max-width: 100%

Determina que a imagem não deve ultrapassar a largura disponível do elemento pai.

11.2 height: auto

Mantém a proporção original da imagem.

Exemplo:

```
  
  
img {  
  max-width: 100%;  
  height: auto;  
}
```

Esse é um dos padrões mais importantes para imagens responsivas.

12. Flexbox e Responsividade

Depois de compreender as unidades relativas, podemos utilizar o Flexbox.

Flexbox é um modelo de layout CSS utilizado principalmente para organizar elementos em uma dimensão:

- linha;
- coluna.

Exemplo:

```
.container {  
  display: flex;  
}
```

HTML:

```
<div class="container">  
  <div class="item">Item 1</div>  
  <div class="item">Item 2</div>  
  <div class="item">Item 3</div>  
</div>
```

CSS:

```
.container {  
  display: flex;  
  gap: 20px;  
}
```

Resultado:

```
+-----+ +-----+ +-----+
| Item 1 | | Item 2 | | Item 3 |
+-----+ +-----+ +-----+
```

13. flex-wrap

Quando os elementos não cabem em uma única linha, podemos permitir que eles sejam deslocados para a próxima linha.

```
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
```

O resultado pode ser:

```
+-----+ +-----+ +-----+
| Item 1 | | Item 2 | | Item 3 |
+-----+ +-----+ +-----+

+-----+
| Item 4 |
+-----+
```

14. Propriedade flex

Podemos permitir que os elementos ocupem o espaço disponível.

```
.item {
  flex: 1;
}
```

Também podemos estabelecer um tamanho mínimo utilizando:

```
.item {
  flex: 1 1 300px;
}
```

A propriedade pode ser entendida como:

```
flex-grow
flex-shrink
flex-basis
```

Nesse caso:

```
flex-grow: 1
flex-shrink: 1
flex-basis: 300px
```

Uma interpretação simples é:

Cada elemento tenta possuir aproximadamente 300px, mas pode crescer ou diminuir conforme o espaço disponível.

15. Exemplo Prático com Flexbox

HTML:

```
<section class="cards">
  <article class="card">
    <h2>HTML</h2>
    <p>Estrutura da página.</p>
  </article>
  <article class="card">
    <h2>CSS</h2>
    <p>Estilização da página.</p>
  </article>
  <article class="card">
    <h2>JavaScript</h2>
    <p>Comportamento da página.</p>
  </article>
</section>
```

CSS:

```
.cards {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
}
.card {
  flex: 1 1 250px;
  padding: 20px;
  border: 1px solid #ccc;
}
```

Em uma tela grande:

```
+-----+ +-----+ +-----+
|   HTML   | |   CSS   | | JavaScript |
+-----+ +-----+ +-----+
```

Em uma tela menor:

```
+-----+ +-----+
|   HTML   | |   CSS   |
+-----+ +-----+

+-----+
| JavaScript |
+-----+
```

16. CSS Grid

O CSS Grid é outro sistema de layout muito importante para responsividade.

Enquanto o Flexbox trabalha principalmente em uma dimensão, o Grid permite trabalhar com:

- linhas;
- colunas.

Exemplo:

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}
```

O resultado será semelhante a:

1	2	3
---	---	---

17. Unidade fr

A unidade fr representa uma fração do espaço disponível no Grid.

Exemplo:

```
grid-template-columns: 1fr 1fr 1fr;
```

O espaço será dividido em três partes iguais.

Também podemos utilizar:

```
grid-template-columns: 2fr 1fr;
```

Nesse caso:

2fr	1fr
-----	-----

18. Grid Responsivo com auto-fit e minmax()

Uma técnica muito útil é:

```
.container {  
  display: grid;  
  grid-template-columns:  
    repeat(auto-fit, minmax(250px, 1fr));  
  gap: 20px;  
}
```

repeat()

Repete a definição de coluna.

auto-fit

Permite ao navegador ajustar automaticamente a quantidade de colunas que cabe no espaço disponível.

minmax()

Define um tamanho mínimo e um tamanho máximo.

Por exemplo:

```
minmax(250px, 1fr)
```

significa que cada coluna deverá tentar possuir pelo menos 250px e poderá ocupar uma fração do espaço disponível.

Essa abordagem permite criar layouts adaptáveis sem necessariamente utilizar várias Media Queries.

19. Media Queries

Media Query é um recurso do CSS que permite aplicar determinadas regras somente quando determinadas condições forem satisfeitas.

Exemplo:

```
@media (max-width: 768px) {  
  .menu {  
    flex-direction: column;  
  }  
}
```

Isso significa:

Quando a largura da viewport for igual ou menor que 768px, aplique essas regras.

20. max-width

Exemplo:

```
@media (max-width: 600px) {  
  body {  
    background-color: lightgray;  
  }  
}
```

A regra será aplicada quando a viewport possuir até 600px de largura.

21. min-width

Também podemos fazer o contrário:

```
@media (min-width: 768px) {  
  .menu {  
    display: flex;  
  }  
}
```

Nesse caso, a regra será aplicada quando a largura da viewport for igual ou superior a 768px.

22. Exemplo Completo com Media Query

HTML:

```
<div class="container">  
  <div class="card">Card 1</div>  
  <div class="card">Card 2</div>  
  <div class="card">Card 3</div>  
</div>
```

CSS:

```
.container {  
  display: flex;  
  gap: 20px;  
}  
.card {  
  flex: 1;  
  padding: 30px;  
  border: 1px solid black;  
}  
  
@media (max-width: 600px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

No desktop:

```
+-----+ +-----+ +-----+  
| Card 1 | | Card 2 | | Card 3 |  
+-----+ +-----+ +-----+
```

No celular:

```
+-----+
| Card 1 |
+-----+
```

```
+-----+
| Card 2 |
+-----+
```

```
+-----+
| Card 3 |
+-----+
```

23. Mobile First

Mobile First é uma estratégia de desenvolvimento em que começamos criando a interface pensando primeiro em telas pequenas.

Depois adicionamos regras para telas maiores.

Exemplo:

```
.menu {
  display: flex;
  flex-direction: column;
}
```

Depois:

```
@media (min-width: 768px) {
  .menu {
    flex-direction: row;
  }
}
```

A lógica é:

```
CELULAR
  ↓
TABLET
  ↓
DESKTOP
```

Em vez de desenvolver primeiro para telas grandes e tentar corrigir posteriormente os problemas existentes em telas pequenas.

24. Por que utilizar Mobile First?

Começar pelo menor espaço disponível força o desenvolvedor a priorizar:

- conteúdo;
- navegação;
- legibilidade;
- usabilidade;
- informações realmente necessárias.

Além disso, podemos adicionar progressivamente regras para telas maiores.

Exemplo:

```
@media (min-width: 768px) {  
    ...  
}
```

e:

```
@media (min-width: 1024px) {  
    ...  
}
```

25. Breakpoints

Breakpoint é um ponto em que o layout precisa mudar para continuar funcionando adequadamente.

Exemplo:

```
@media (min-width: 768px) {  
    ...  
}
```

Nesse caso, 768px é um breakpoint.

É importante compreender que breakpoints não devem ser escolhidos simplesmente porque determinados dispositivos possuem determinadas larguras.

O ideal é observar quando o layout deixa de funcionar adequadamente e criar o breakpoint de acordo com essa necessidade.

26. Exemplo com Diferentes Breakpoints

```
/* Configuração inicial: celular */  
  
.container {  
    display: grid;  
    grid-template-columns: 1fr;  
}  
  
/* A partir de 768px */  
  
@media (min-width: 768px) {  
    .container {  
        grid-template-columns: 1fr 1fr;  
    }  
}  
  
/* A partir de 1024px */  
  
@media (min-width: 1024px) {  
    .container {  
        grid-template-columns: 1fr 1fr 1fr;  
    }  
}
```

No celular:

```
+-----+
|     1     |
+-----+
|     2     |
+-----+
|     3     |
+-----+
```

No tablet:

```
+-----+ +-----+
|     1     | |     2     |
+-----+ +-----+
|     3     |
+-----+
```

No desktop:

```
+-----+ +-----+ +-----+
|     1     | |     2     | |     3     |
+-----+ +-----+ +-----+
```

27. Tipografia Responsiva

O tamanho dos textos também precisa acompanhar o dispositivo.

Podemos utilizar a função `clamp()`.

```
h1 {
  font-size: clamp(2rem, 5vw, 4rem);
}
```

A estrutura é:

`clamp(MÍNIMO, VALOR_PREFERENCIAL, MÁXIMO)`

No exemplo:

```
mínimo      = 2rem
preferencial = 5vw
máximo      = 4rem
```

O navegador tenta utilizar o valor preferencial, mas respeita os limites mínimo e máximo.

28. Exemplo de Tipografia Responsiva

```
h1 {
  font-size: clamp(2rem, 6vw, 4rem);
}
p {
  font-size: clamp(1rem, 2vw, 1.25rem);
}
```

Esse recurso permite criar uma tipografia mais fluida entre diferentes tamanhos de tela.

29. Menu Responsivo

Um exemplo clássico de responsividade é o menu de navegação.

Em uma tela grande:

INÍCIO | SOBRE | SERVIÇOS | CONTATO

Em uma tela pequena:



Podemos começar com:

```
.menu {  
    display: none;  
}  
  
@media (min-width: 768px) {  
    .menu {  
        display: flex;  
    }  
}
```

O botão do menu pode aparecer em telas menores:

```
.botao-menu {  
    display: block;  
}  
  
@media (min-width: 768px) {  
    .botao-menu {  
        display: none;  
    }  
}
```

Para tornar o menu realmente interativo, posteriormente podemos utilizar JavaScript.

30. Componentes Responsivos

Depois de aprender a adaptar a página inteira, é importante aprender a pensar em componentes.

Exemplos:

- cards;
- menus;
- formulários;
- tabelas;
- galerias;
- imagens;
- cabeçalhos;
- rodapés.

Exemplo de cards utilizando Grid:

```
.cards {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}
```

Cada card poderá se adaptar automaticamente ao espaço disponível.

31. Formulários Responsivos

HTML:

```
<form>
  <label for="nome">Nome</label>
  <input type="text" id="nome">
  <label for="email"> E-mail </label>
  <input type="email" id="email">
  <button type="submit"> Enviar </button>
</form>
```

CSS:

```
form {
  width: 90%;
  max-width: 600px;
  margin: auto;
}
input {
  width: 100%;
  box-sizing: border-box;
}
```

O formulário utilizará uma porcentagem da largura disponível em telas pequenas, mas terá uma largura máxima em telas grandes.

32. Tabelas Responsivas

Tabelas podem apresentar problemas em smartphones porque muitas colunas podem não caber na largura disponível.

Podemos criar um container:

```
<div class="tabela-container">
  <table>
    ...
  </table>
</div>
```

E utilizar:

```
.tabela-container {
  overflow-x: auto;
}
```

Quando a tabela for maior que o espaço disponível, será possível realizar rolagem horizontal dentro do container.

33. Testando a Responsividade

Não basta escrever o CSS. É necessário testar o comportamento da página.

Nos navegadores baseados em Chromium, podemos abrir as ferramentas de desenvolvimento utilizando:

F12

Também podemos utilizar:

Ctrl + Shift + M

para ativar o modo de dispositivos.

Podemos testar diferentes larguras, como:

390px
768px
1024px
1280px
1920px

Durante os testes devemos verificar:

- existe rolagem horizontal?
 - algum elemento ultrapassa a tela?
 - o texto está legível?
 - os botões são fáceis de utilizar?
 - as imagens estão adequadas?
 - o menu funciona?
 - os cards estão organizados?
 - existe conteúdo cortado?
 - existem elementos sobrepostos?
 - existe espaço exagerado entre elementos?
-

34. Projeto Prático Final

Depois de estudar os conceitos, recomenda-se desenvolver uma página Web completa e responsiva.

Um exemplo de estrutura:



O projeto deverá funcionar adequadamente em:

Desktop
↓
Tablet
↓
Smartphone

35. Exemplo de Projeto Integrado

HTML

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Site Responsivo</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Minha Empresa</h1>
    <nav>
      <a href="#">Início</a>
      <a href="#">Sobre</a>
      <a href="#">Serviços</a>
      <a href="#">Contato</a>
    </nav>
  </header>
  <main>
    <section class="hero">
```

```

    <h2> Desenvolvimento Web </h2>
    <p> Criando interfaces modernas e responsivas. </p>
</section>
<section class="cards">
  <article class="card">
    <h3>HTML</h3>
    <p> Estrutura das páginas Web. </p>
  </article>
  <article class="card">
    <h3>CSS</h3>
    <p> Estilização e layout. </p>
  </article>
  <article class="card">
    <h3>JavaScript</h3>
    <p> Interatividade e comportamento. </p>
  </article>
</section>
</main>
<footer>
  <p> © 2026 Minha Empresa </p>
</footer>
</body>
</html>

```

CSS

```

* {
  box-sizing: border-box;
}
body {
  margin: 0;
  font-family: Arial, sans-serif;
}
header {
  padding: 1rem;
  text-align: center;
}
nav {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}
nav a {
  text-decoration: none;
}
.hero {
  width: 90%;
  min-height: 40vh;
  margin: auto;
  padding: 2rem;
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  text-align: center;
}
.hero h2 {
  font-size: clamp(2rem, 6vw, 4rem);
}
.cards {
  width: 90%;

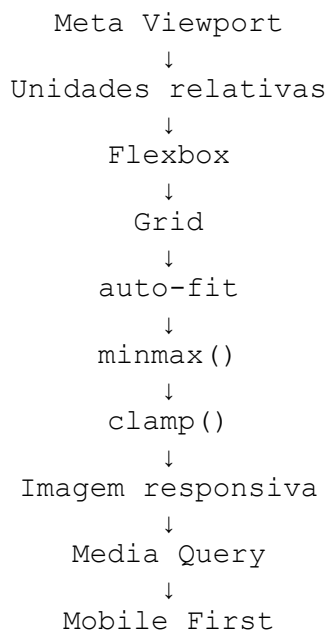
```

```

margin: auto;
display: grid;
grid-template-columns:
    repeat(auto-fit, minmax(250px, 1fr));
gap: 20px;
}
.card {
padding: 2rem;
border: 1px solid #ccc;
}
img {
max-width: 100%;
height: auto;
}
footer {
margin-top: 2rem;
padding: 2rem;
text-align: center;
}
@media (min-width: 768px) {
nav {
flex-direction: row;
justify-content: center;
}
}
}

```

Esse projeto reúne diversos conceitos estudados:



36. Principais Comandos e Recursos

Recurso	Exemplo	Finalidade
%	width: 80%	Tamanho relativo
rem	font-size: 2rem	Tamanho relativo ao elemento raiz
em	padding: 1em	Tamanho relativo ao contexto
vw	width: 50vw	Relativo à largura da viewport
vh	height: 50vh	Relativo à altura da viewport
max-width	max-width: 100%	Impede que o elemento ultrapasse o container

display: flex	display: flex	Ativa o Flexbox
flex-wrap	flex-wrap: wrap	Permite quebra de linha
flex	flex: 1 1 300px	Controla crescimento, redução e tamanho inicial
display: grid	display: grid	Ativa o CSS Grid
fr	1fr 1fr	Divide o espaço disponível
minmax()	minmax(250px, 1fr)	Define tamanho mínimo e máximo
auto-fit	repeat(auto-fit, ...)	Ajusta automaticamente as colunas
@media	@media (max-width: 600px)	Adapta regras conforme a viewport
clamp()	clamp(1rem, 3vw, 2rem)	Cria valores fluidos com limites
overflow-x	overflow-x: auto	Permite rolagem horizontal quando necessária

37. Sequência de Aprendizagem

A aprendizagem da responsividade pode seguir a seguinte sequência:

1. Entender o problema
- ↓
2. Viewport
- ↓
3. Meta viewport
- ↓
4. Unidades relativas
- ↓
5. Imagens responsivas
- ↓
6. Flexbox
- ↓
7. Grid
- ↓
8. Media Queries
- ↓
9. Mobile First
- ↓
10. Breakpoints
- ↓
11. Tipografia responsiva
- ↓
12. Menus e componentes
- ↓
13. Formulários e tabelas
- ↓
14. Testes no navegador
- ↓
15. Projeto completo